Classification Model

- Classification: Basic Concepts
- Rule-Based Classification
- Decision Tree Induction
- Bayes Classification Methods
- kNN Classifier
- Performance Evaluation
- Ensemble Classification
- Summary

Model Evaluation and Selection

- Evaluation metrics: How can we measure accuracy? Other metrics to consider?
- Use **validation test set** of class-labeled tuples instead of training set when assessing performance of the classifiers
- Methods for estimating a classifier's performance:
  - Holdout method, random subsampling
  - Bootstrap
  - Cross-validation

Holdout Method

- The holdout method is a data splitting technique that divides the original dataset into two subsets such as, training set and test set.
- The training set is used to build and train the classifier, while the test set is used to evaluate how well the model can generalize to new data.
- The test set is also called the holdout set, because it is held out from the model development process.
- To apply the holdout method, you need to decide how to split the data into training and test sets.

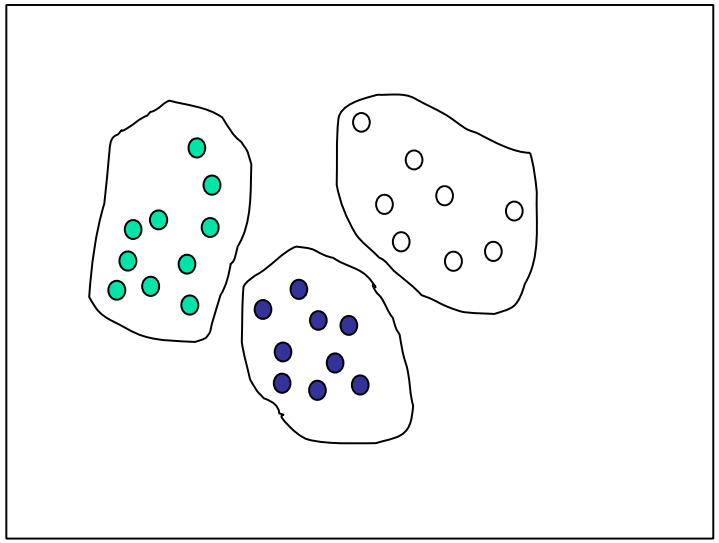
Holdout Method

- A common rule of thumb is to use two-thirds of the data for training and one-third for testing.
- But this may vary depending on the size and characteristics of the data.
- The model is trained on training data and evaluated on test data.
- Here, we need to ensure that the two sets are representative of the original data, meaning that they have similar distributions and proportions of objects and classes.
- This can be done by stratified sampling, which preserves the relative frequencies of different groups in the data.

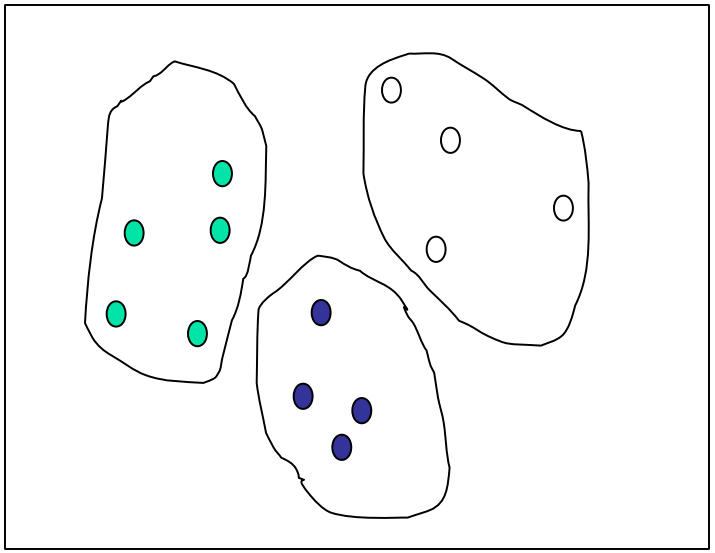
Sampling

- Sampling: obtaining a small sample *s* to represent the whole data set *N*
- Stratified sampling: Approximate the percentage of each class in the overall database

Raw Data



Stratified Sample



Limitations

i) Fewer objects are available for training as some of the objects are withheld for testing. As a result, the trained model may not be as good as when all the labeled objects are used for training.

ii) The model may be highly dependent on the composition of the training and test sets.

- The smaller the training set size, the larger the variance of the model.

- On the other hand, if the training set is too large, then the estimated test accuracy is less reliable. With a very large training set, the model may become overly specialized in recognizing patterns in the training data, making it less generalizable to unseen test data (**Overfitting to Training Data**).

Advantages

- It is simple and easy to implement, requiring only one split of the data.
- It is efficient and fast, as it only requires one run of the model on the test set.

Disadvantages

- The holdout method also has some disadvantages that limit its reliability and usefulness.
- It is sensitive to the choice of the split, as different splits may result in different model performance and accuracy.
- It is wasteful and potentially biased, as it does not use all the available data for training or testing.
- It is unstable and variable, as it may not reflect the true performance of the model over multiple runs.

Random subsampling

- The holdout method can be repeated several times to improve the estimation of the classifier's performance. This approach is known as random subsampling.
- Let Acc_i be the model accuracy during the i-th iteration. The overall accuracy is given by $A = \sum_{i=1}^{k} \frac{Acc_i}{k}$
- Random sampling still encounters some of the problems associated with the holdout method because it does not utilize as much data as possible for training.
- It also has no control over the number of times each record is used for testing and training. Consequently, some objects might be used for training more often than others.

Bootstrap

- The methods presented so far assume that the training records are sampled without replacement. As a result, there are no duplicate records in the training and test sets.
- **Bootstrap** is a technique that resamples the data with replacement.
- This means that some data points may be included in the resampled dataset multiple times, while others may not be included at all.
- The model is then trained on the resampled dataset and evaluated on the test dataset (not in the bootstrap sample).
- This process is repeated multiple times, and the results are averaged to get an estimate of the model's performance.

Bootstrap

- **Bootstrap**
  - Works well with small data sets
  - Samples the given training tuples uniformly *with replacement*
    - i.e., each time a tuple is selected, it is equally likely to be selected again and re-added to the training set
- There are several bootstrap methods, and a common one is **0.632 boostrap**
- **0.632 boostrap:**
  - Let D is a data set with size |D|=N. Let dataset D_1 is the bootstrap sample set with same size, N. Here, random sampling with replacement is used to make D_1 from D. So some samples may appear many times and some may not appear in D_1.
  - The bootstrap samples (i,.e., D_1) is used as training set and the samples of D not included in D_1 is the test set, say T_1.

Bootstrap

- On average, a bootstrap sample set, (i.e. D_1), contains approximately 63% samples of the original dataset because, each sample of D has a probability $1-(1-\frac{1}{N})^N$ of being selected in D_1. How?

  - Probability that a sample is selected in D_1 = $\frac{1}{N}$.

  - Therefore, probability that the sample is not selected in D_1 = $1-\frac{1}{N}$

  - Since, we are using sampling with replacement, so for second time sample selection in D_1, probability that the sample is not selected in D_1 is also = $1-\frac{1}{N}$

  - Since, the size of D_1 is also = N, so the probability that a sample of D is not selected = $(1-\frac{1}{N})^N$

Bootstrap

- Therefore, the probability of each sample of D is selected in D_1 = $1 - (1 - \frac{1}{N})^N$
- If N is sufficiently large, this probability converges to $1 - \frac{1}{e} \simeq 0.632$
- The samples of D, which are not in D_1 is the test set, T_1.
- Repeat the sampling procedure *k* times, we get the models, M_1, M_2, …, M_k, where, the training and test set for M_i are D_i and T_i respectively.
- Therefore, the overall accuracy of the model:

$$\text{Accuracy}_{0.632} = 0.632 \times \text{Accuracy}_{\text{test}} + (1 - 0.632) \times \text{Accuracy}_{\text{train}}$$

- $\text{Accuracy}_{\text{test}}$ = average accuracy on test sets
- $\text{Accuracy}_{\text{train}}$ = average accuracy on the training sets.

Bootstrap

- In holdout method, the test data is **never seen by the model** during training, its accuracy gives an **unbiased estimate of generalization performance**. That is why, only test accuracy is considered.
- In the **holdout method**, each data point is used **only once** (either for training or testing).
- In **bootstrap**, the same data points are **reused** in different bootstrap samples, so same samples are reused in different test set which can lead to bias in test accuracy.
- To correct for this bias, bootstrap uses the **0.632 rule**, incorporating training accuracy to balance the estimate.

Why used both Training or Test Accuracy?

- The **0.632 accuracy equation** is used because it balances the **bias** from training accuracy and the **variance** from test accuracy, providing a more reliable estimate of a classifier's performance.
- **Training accuracy ($\text{Accuracy}_{\text{train}}$)** is **overoptimistic** because the classifier is tested on the same data it was trained on.
- **Test accuracy ($\text{Accuracy}_{\text{test}}$)** from test samples is **more realistic** but is based on a **subset (36.8%)** of the data, which introduces higher variance.
- Thus, relying on either alone is not ideal.

Why the 0.632 Weight?

- Since **not all data points are used for testing**, relying only on test accuracy would result in **high variance** due to limited test data.
- The training accuracy is considered to **compensate for the bias** in the test accuracy.
- To balance the contributions of both training and test accuracy, the equation gives:
  - **63.2% weight to the test accuracy** (since it better estimates generalization performance).
  - **36.8% weight to training accuracy** (to compensate for the reduced training set size in each bootstrap iteration).

Bootstrap

1. **Advantages**

- Reduces the **overfitting bias** of training accuracy.
- Provides a better accuracy estimate compared to simple holdout validation.
- More stable than k-fold cross-validation for small datasets.

2. **Limitations**

- May still have some bias, especially if the classifier overfits to the training data.
- Computationally expensive for large datasets, as multiple bootstrap samples are required.

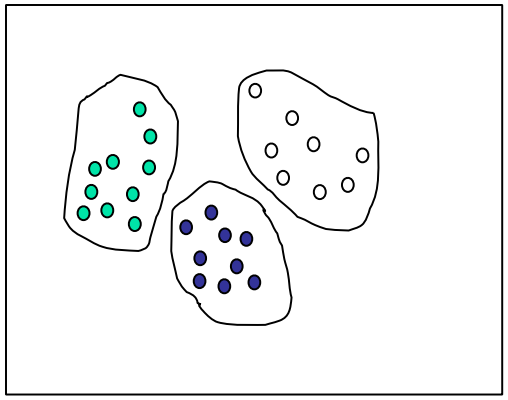
Cross-validation

- **K-fold Cross-validation** (*k*-fold, where k = 10 is most popular)
  - Randomly partition the data into *k mutually exclusive* subsets, each approximately equal size
  - At *i*-th iteration, use D_i as test set and others as training set
  - Leave-one-out: *k* folds where *k* = # of tuples, for small sized data

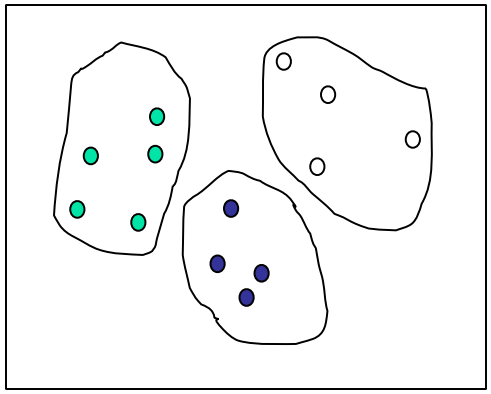
Cross validation with k = 3

Cross-validation

- **Stratified cross-validation**: folds are stratified so that class distribution in each fold is approximately the same as that in the initial data



original data



Stratified sample

Classifier Evaluation Metrics: Confusion Matrix

- A confusion matrix, as the name suggests, is a matrix of numbers that tell us where a model gets confused.
- It is a class-wise distribution of the predictive performance of a classification model, i.e., the confusion matrix is an organized way of mapping the predictions to the original classes to which the data belong.
- A binary class dataset is one that consists of just two distinct categories of data. These two categories can be named the “positive” and “negative” for the sake of simplicity.

Confusion Matrix for Binary Classifier

- The basic classifier evaluation metrics are: Accuracy, Precision, Recall, and F-Measure
- These metrics are defined by the following terms:

i. True Positive (TP): Positive class correctly classified as positive

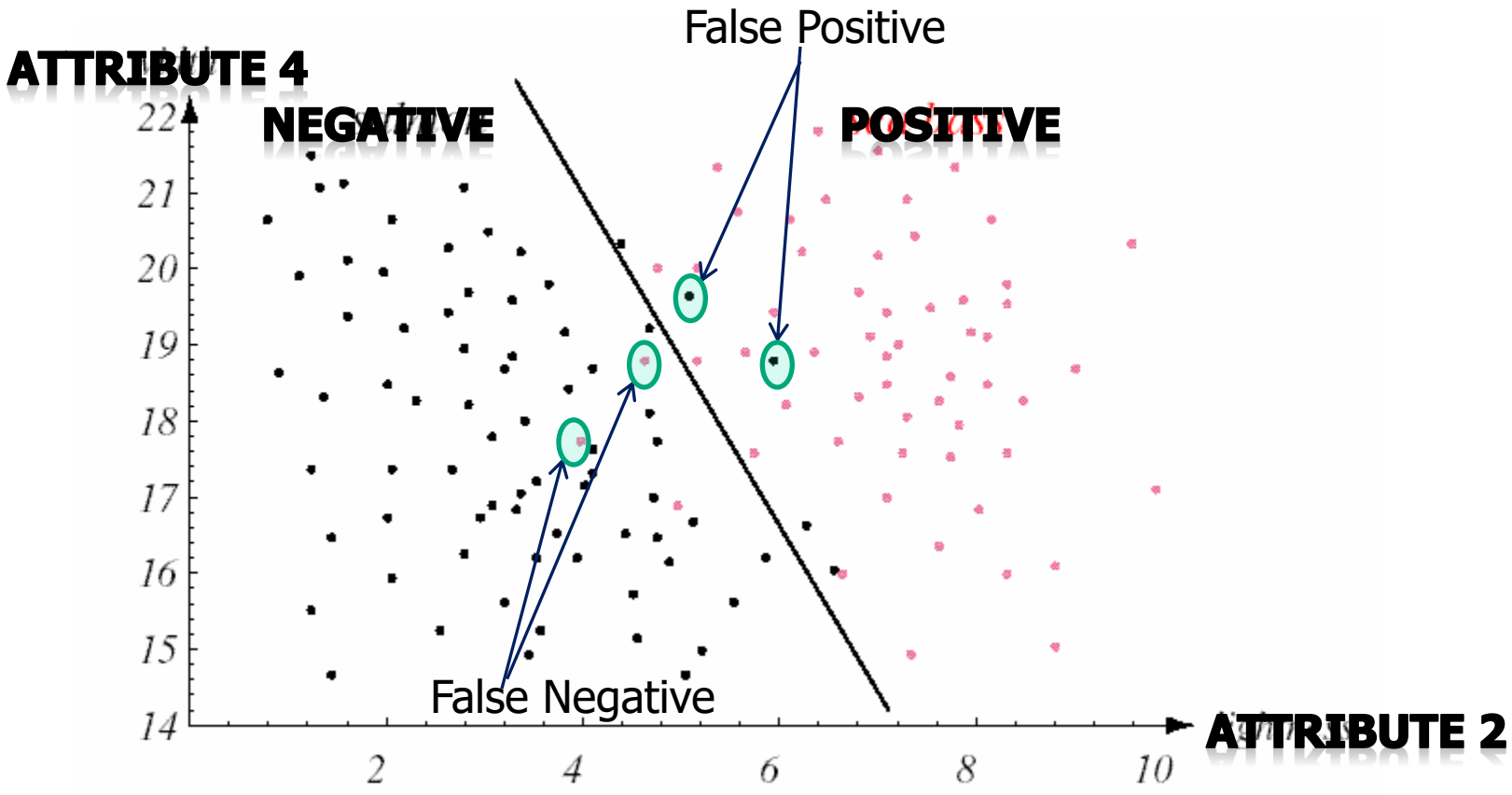
ii. False positive (FP): Negative class incorrectly classified as positive

iii. True Negative (TN): Negative class correctly classified as negative

iv. False Negative (FN): Positive class incorrectly classified as negative

- Usually, the larger class is the negative class

Classifier Evaluation Metrics: Confusion Matrix



Classifier Evaluation Metrics: Confusion Matrix

Confusion Matrix	Predicted	
	Positive	Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

- **Classifier Accuracy,** or recognition rate: percentage of test set tuples that are correctly classified.
  - Accuracy (A) = (TP+TN) / (TP+ TN + FP + FN)
  - Error Rate (E) = 1 – Accuracy = (FP+FN) / (TP+ TN + FP + FN)

Classifier Evaluation Metrics: Confusion Matrix

- In many cases, accuracy may be very high but the model is not useful, in case of class imbalanced data set.
- Let in disease data set, positive samples are very few compare to negative samples.
- If the model predict most of the negative samples correctly but predict correctly very few of the positive samples, then though the accuracy is very high still the model is not useful.
- In this situation, we use other performance metrics, such as precision, recall, etc.

Classifier Evaluation Metrics: Confusion Matrix

Confusion Matrix	Predicted
	Positive Negative
Actual Positive	True Positive (TP) False Negative (FN)
Actual Negative	False Positive (FP) True Negative (TN)

- **Precision**: exactness – what % of tuples that the classifier labeled as positive are actually positive?

$$precision = \frac{TP}{TP + FP}$$

- **Recall:** completeness – what % of positive tuples did the classifier label as positive?

$$recall = \frac{TP}{TP + FN}$$

- Perfect score of Precision and Recall is 1.0
- There is an inverse relationship between precision & recall

Classifier Evaluation Metrics: Confusion Matrix

Confusion Matrix	Predicted Positive Negative
Positive **Actual** Negative	True Positive (TP) False Negative (FN) False Positive (FP) True Negative (TN)

- ***F-M*easure (*F_1* or *F*-score)**: harmonic mean of precision and recall.

$$F = \frac{2 \times precision \times recall}{precision + recall}$$

Classifier Evaluation Metrics: Confusion Matrix

- F_{β}: weighted measure of precision and recall. It assigns ß times as much weight to recall as to precision
- It means that recall is considered **ß times more important** than precision
- **If ß > 1**, recall is more important than precision.
- **If ß < 1**, precision is more important than recall.
- **If ß = 1**, it becomes the standard **F1-score**, where precision and recall are equally weighted

$$F\beta \text{ Score} = \frac{1+\beta^2}{\frac{1}{\text{Precision}} + \frac{\beta^2}{\text{Recall}}}$$

$$= \frac{(1+\beta^2) \times \text{Precision} \times \text{Recall}}{(\beta^2 \times \text{Precision}) + \text{Recall}}$$

Classifier Evaluation Metrics: Sensitivity and Specificity

Confusion Matrix	Predicted
	Positive Negative
Positive **Actual** Negative	True Positive **(TP)** False Negative **(FN)**
	False Positive **(FP)** True Negative **(TN)**

- **Sensitivity**: True Positive recognition rate = $\frac{TP}{P} = \frac{TP}{TP + FN}$ = **Recall**
- **Specificity**: True Negative recognition rate = $\frac{TN}{N} = \frac{TN}{TN + FP}$

Actual Class \ Predicted class	cancer = yes	cancer = no	Total	Recognition(%)
cancer = yes	**90**	**210**	300	Sensitivity = 30.00
cancer = no	**140**	**9560**	9700	Specificity = 98.56
Total	230	9770	10000	Accuracy=96.40
Precision=90/230 = 39.13,		Recall=90/300=30.00,		F-Score=33.96

Confusion Matrix for Multiple Classes

- The concept of the multi-class confusion matrix is similar to the binary-class matrix. Let, the columns represent the actual class, and the rows represent the predicted class distribution by the classifier.
- Let us elaborate on the features of the multi-class confusion matrix with an example.
- Suppose we have the test set (consisting of 191 total samples) of a dataset with the following distribution:

Class	Nº of Samples
1	60
2	34
3	43
4	54

Confusion Matrix for Multiple Classes

- The confusion matrix obtained by training a classifier and evaluating the trained model on this test set is shown below.

Predicted \ Expected	1	2	3	4
1	52	3	7	2
2	2	28	2	0
3	5	2	25	12
4	1	1	9	40

- Let that matrix be called "*M*," and each element in the matrix be denoted by M_{ij} , where "*i*" is the row number (predicted class), and "*j*" is the column number (Actual or expected class), e.g., M_{11}=52, M_{42}=1.

Confusion Matrix for Multiple Classes

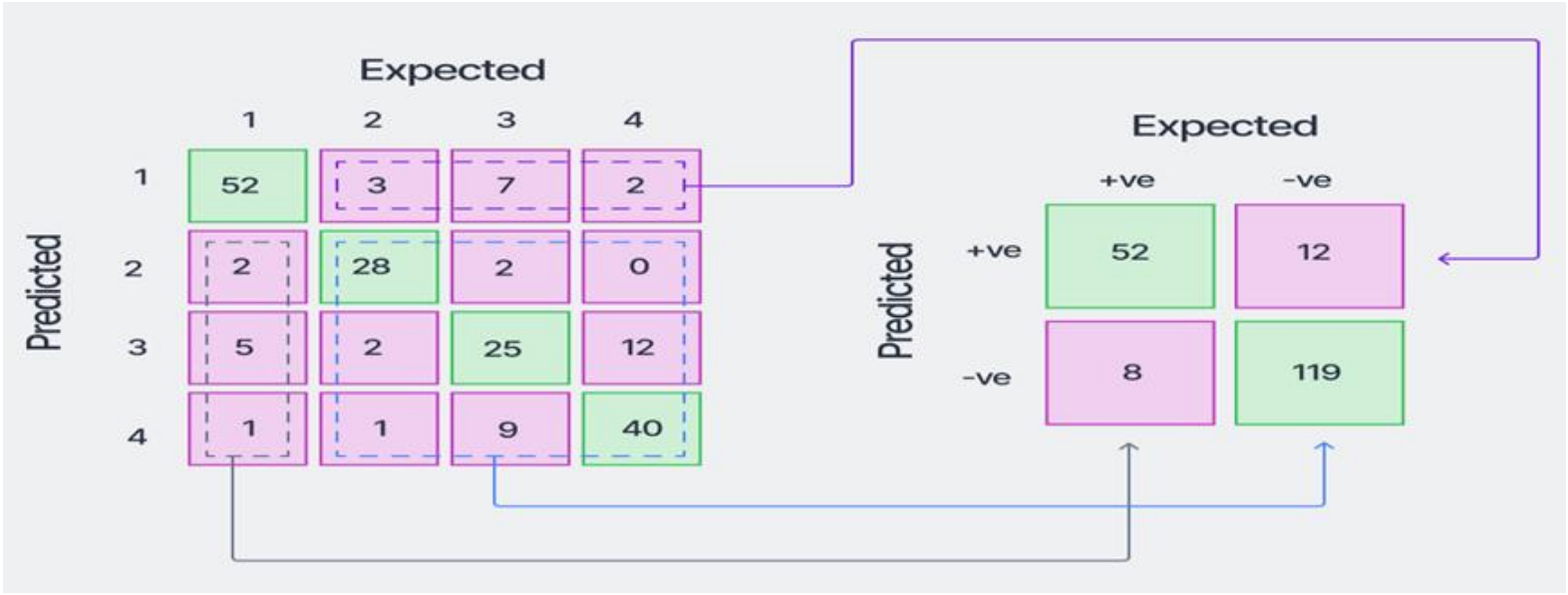
- This confusion matrix gives a lot of information about the model's performance.
- As usual, the diagonal elements are the correctly predicted samples. A total of 145 samples were correctly predicted out of the total 191 samples. Thus, the overall accuracy is 75.92%.
- M_{24}=0 implies that the model does not confuse samples originally belonging to class-4 with class-2, i.e., the classification boundary between classes 2 and 4 was learned well by the classifier.

Confusion Matrix for Multiple Classes

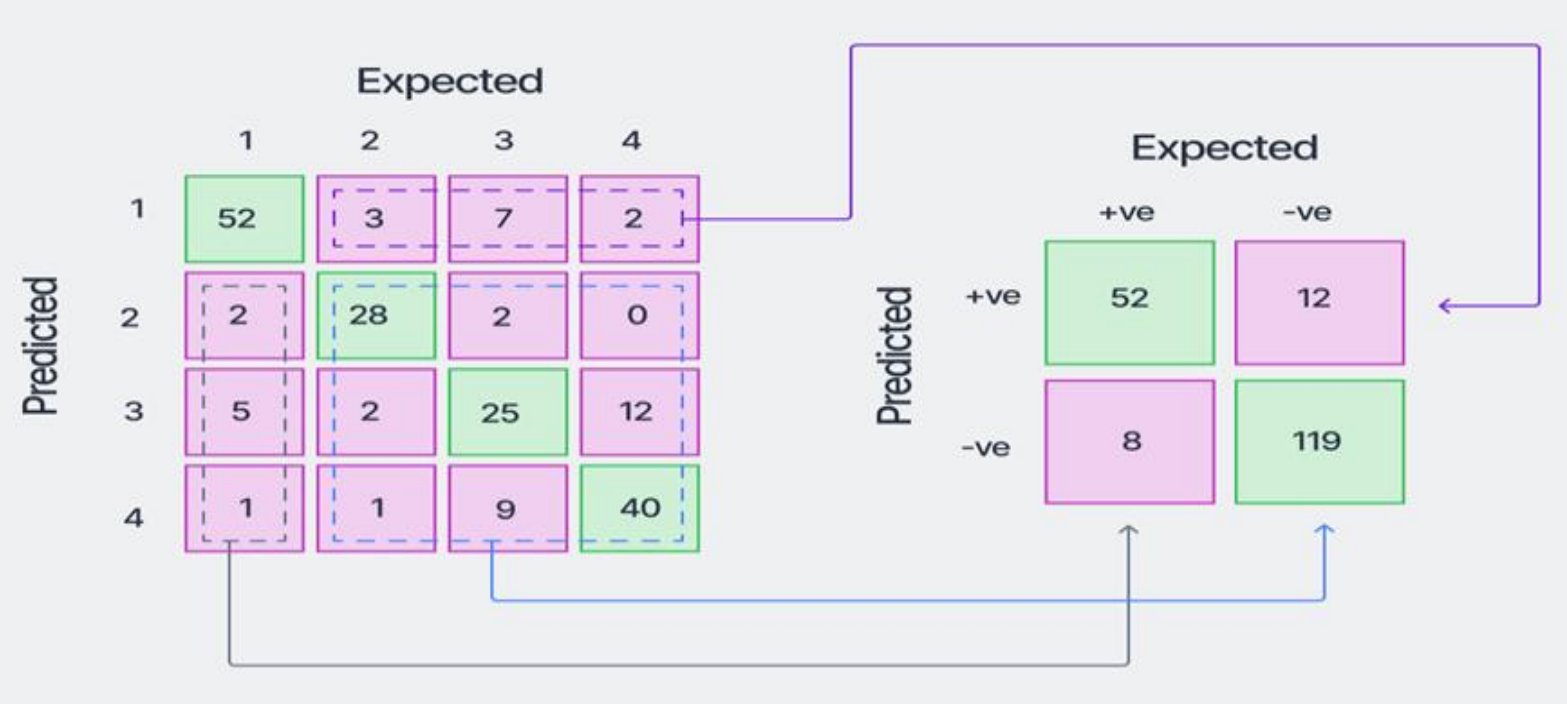
- To improve the model's performance, one should focus on the predictive results in class-3.
- A total of 18 samples (adding the numbers in the red boxes of column 3) were misclassified by the classifier, which is the highest misclassification rate among all the classes.
- Accuracy in prediction for class-3 is, thus, 25/(25+18)=58.14% only.

Confusion Matrix for Multiple Classes

- The confusion matrix can be converted into a one-vs-all type matrix (binary-class confusion matrix) for calculating class-wise metrics like accuracy, precision, recall, etc.
- Converting the matrix to a one-vs-all matrix for class-1 of the data looks like as shown below.



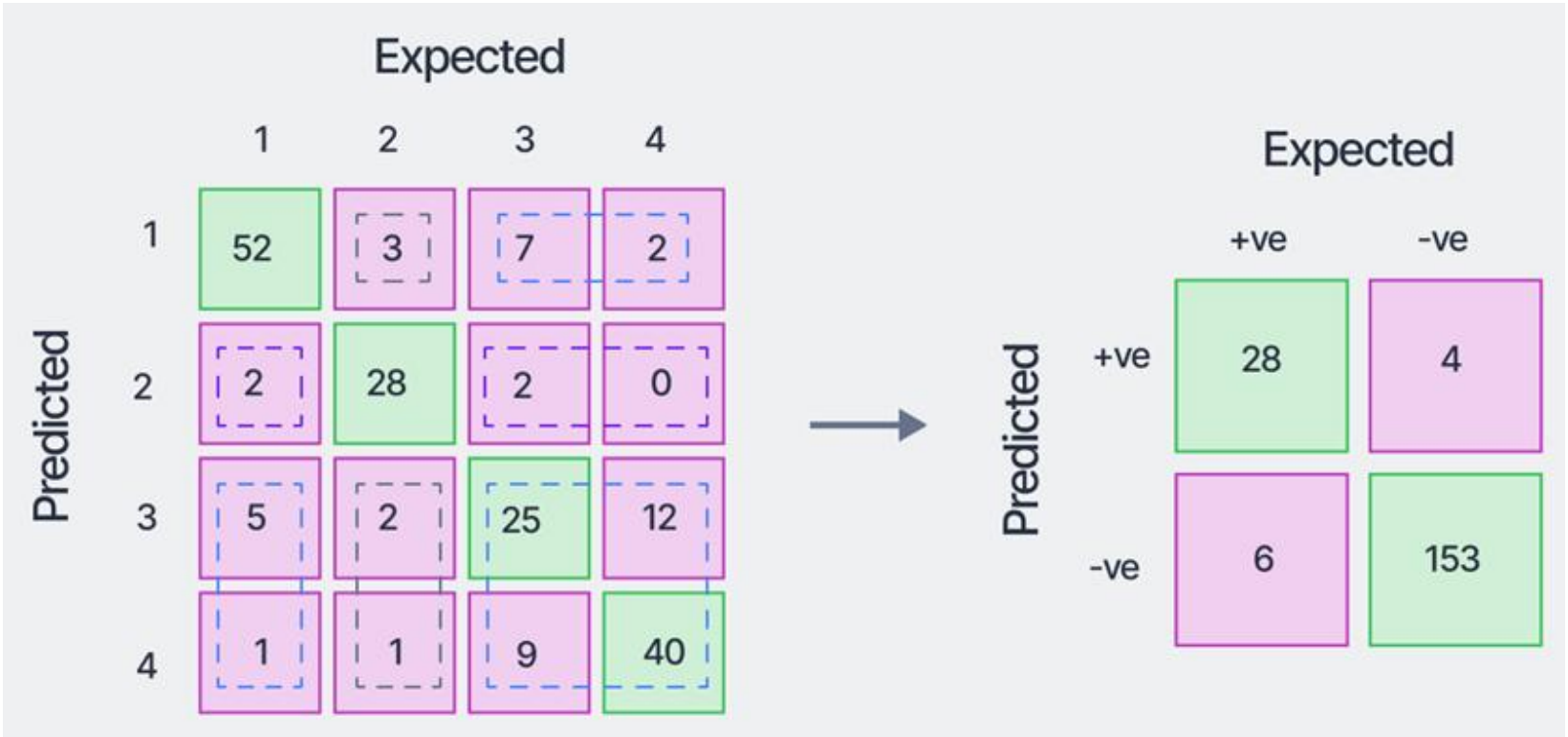
Confusion Matrix for Multiple Classes



• Here, the positive class refers to class-1, and the negative class refers to "NOT class-1". Now, the formulae for the binary-class confusion matrices can be used for calculating the class-wise metrics.

Confusion Matrix for Multiple Classes

- Similarly, for class-2, the converted one-vs-all confusion matrix will look like the following:



Confusion Matrix for Multiple Classes

- Using this concept, we can calculate the class-wise accuracy, precision, recall, and f1-scores and tabulate the results:

Class	Precision (%)	Recall (%)	F1-Score (%)
1	81. 25	86. 67	83. 87
2	87. 50	82. 35	84. 85
3	56. 82	58. 14	57. 47
4	78. 43	74. 07	76. 19

Confusion Matrix for Multiple Classes

- In addition to these, two more global metrics can be calculated for evaluating the model's performance over the entire dataset.
- These metrics are variations of the F1-Score we calculated earlier.
- These are:

  i) Micro-averaged F1-score, and

  ii) Macro-averaged F1-Score

Micro-averaged F1-score

- The micro-averaged F1-score is a global metric that is calculated by considering the

  i) Net TP, i.e., the sum of the class-wise TP (from the respective one-vs-all matrices),

  ii) Net FP, and

  iii) Net FN

- These are obtained as follows:

  Net TP = 52+28+25+40 = 145
  Net FP = (3+7+2)+(2+2+0)+(5+2+12)+(1+1+9) = 46
  Net FN = (2+5+1)+(3+2+1)+(7+2+9)+(2+0+12) = 46

- Note that, the net FP and net FN have the same value.

Micro-averaged F1-score

- Thus, the micro precision and micro recall can be calculated as:

  i) Micro Precision = Net TP/(Net TP+Net FP) = 145/(145+46) = 75.92%
  ii) Micro Recall = Net TP/(Net TP+Net FN) = 75.92%
- So, Micro F1-score = Harmonic Mean of Micro Precision and Micro Recall = 75.92%.
- Here, Micro Precision = Micro Recall = Micro F1-Score = Accuracy = 75.92%

Macro unweighted average score

- **Macro averaged F1-Score:** The macro-average scores are calculated for each class individually, and then the unweighted mean of the measures is calculated to calculate the net global score.
- For the example we have been using, the scores are obtained as the following:
- The unweighted means of the measures are:
- Macro Precision = 76.00%
  Macro Recall = 75.31%
  Macro F1-Score = 75.60%

Class	Precision (%)	Recall (%)	F1-Score (%)
1	81.25	86.67	83.87
2	87.50	82.35	84.85
3	56.82	58.14	57.47
4	78.43	74.07	76.19

Macro weighted average score

- The macro weighted-average scores take a sample-weighted mean of the class-wise scores obtained.
- So, the macro weighted scores obtained are:

$$Weighted\ Precision = \frac{81.25x\mathbf{60} + 87.50x\mathbf{34} + 56.82\ x\ \mathbf{43} + 78.43\ x\ \mathbf{54}}{\mathbf{60} + \mathbf{34} + \mathbf{43} + \mathbf{54}}\% = 76.07\%$$

$$Weighted\ Recall = \frac{86.67x\mathbf{60} + 82.35x\mathbf{34} + 58.14\ x\ \mathbf{43} + 74.07\ x\ \mathbf{54}}{\mathbf{60} + \mathbf{34} + \mathbf{43} + \mathbf{54}}\% = 75.92\%$$

$$Weighted\ F1 - Score = \frac{83.87x\mathbf{60} + 84.85x\mathbf{34} + 57.47\ x\ \mathbf{43} + 76.19\ x\ \mathbf{54}}{\mathbf{60} + \mathbf{34} + \mathbf{43} + \mathbf{54}}\% = 75.93\%$$

Receiver Operating Characteristics (ROC) curve

- A Receiver Operating Characteristics (ROC) curve is a plot for displaying the tradeoff between “true positive rate” and “false positive rate” of a classifier at different threshold settings.
- ROC curves are usually defined for a binary classification model, although that can be extended to a multi-class setting.
- The definition of the true positive rate (TPR) coincides exactly with the sensitivity (or recall).
- TPR is defined as the number of samples belonging to the positive class of a dataset, being classified correctly by the predictive model.
- So the formula for computing the TPR is,

$$TPR = \frac{TP}{TP + FN} = \text{Recall}$$

Receiver Operating Characteristics (ROC) curve

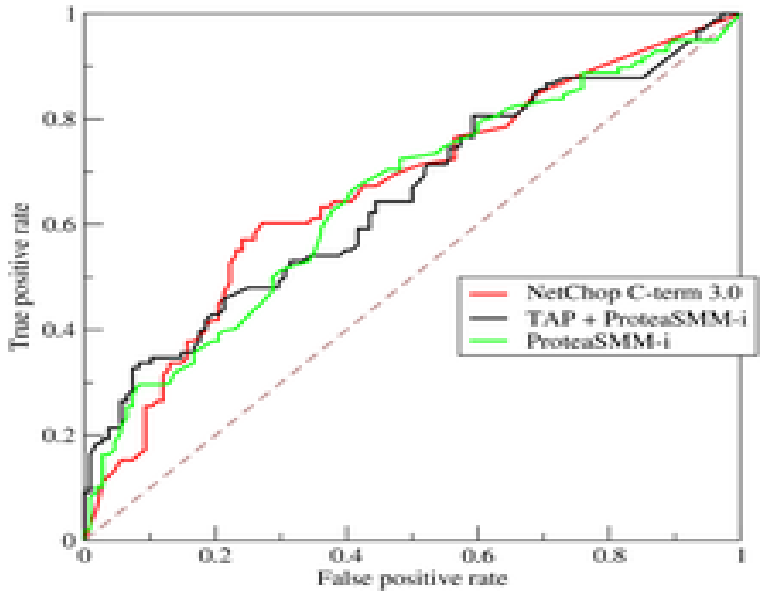
- The false positive rate (FP) is defined as the number of negative class samples predicted *wrongly* to be in the positive class (i.e., the False Positives), out of all the samples in the dataset that actually belong to the negative class.
- Mathematically it is represented as the following:

$$FPR = \frac{FP}{FP + TN}$$
$$= 1 - \frac{TN}{FP + TN}$$
$$= 1 - \text{Specificity}$$

- Note that, the FPR is the additive inverse of Specificity . So both the TPR and FPR can be computed easily from our existing computations from the Confusion Matrix.

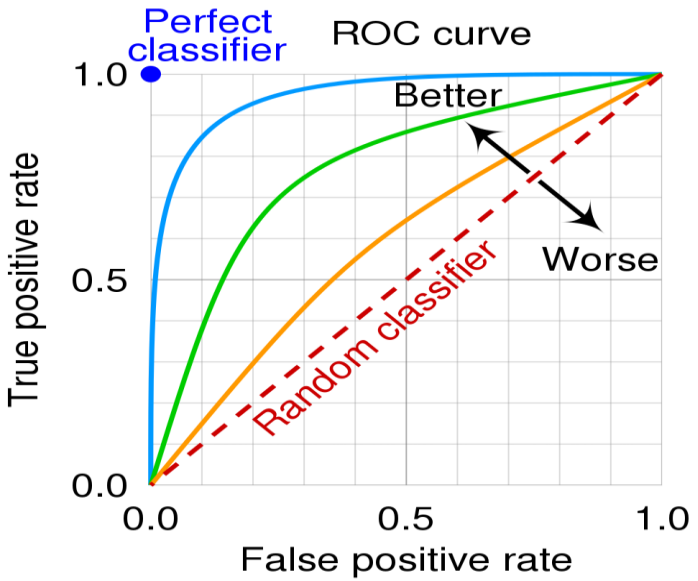
ROC curve

- In a ROC curve, TPR is plotted along the y-axis and the FPR is plotted along the x-axis.
- Each point along the curve corresponds to one of the models induced by the classifier.
- Figure shows the ROC curves for three different classifiers.



ROC Curve

- There are several critical points along an ROC curve that have well-known interpretations:



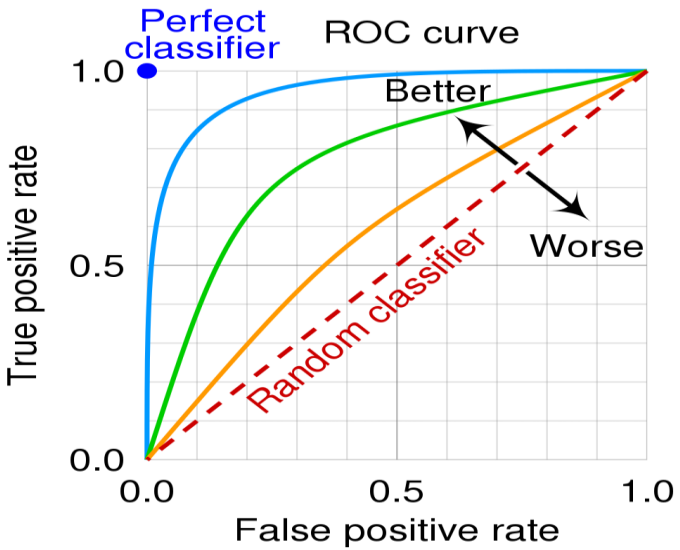
i) (TPR=0, FPR=0): Model predicts every instance to be a negative class.

ii) (TPR=1, FPR=1): Model predicts every instance to be a positive class.

iii) (TPR=1, FPR=0): The ideal model.

ROC Curve

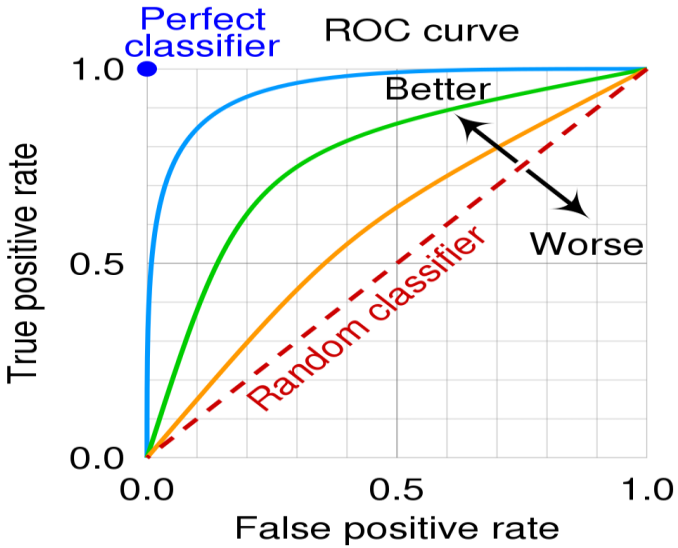
- A good classifier should be located as close as possible to the upper left corner of the diagram.
- A model that makes random predictions is called a Random classifier or "No Skill" classifier.



This model should reside along the main diagonal, connecting the points (TPR=0, FPR=0) and (TPR=1, FPR=1).

- In Random classifier, for a class-balanced dataset, the class-wise probabilities will be 50%. It acts as a reference line for the plot of the precision-recall curve.

ROC Curve



- A perfect learner is one which classifies every sample correctly, and it also acts as a reference line for the ROC plot.
- A real-life classifier will have a plot somewhere in between these two reference lines.
- The more a ROC of a learner is shifted towards the (TPR=0, FPR=1) point (i.e., towards the perfect learner curve), the better is its predictive performance across all thresholds.

ROC Curve

- Another important metric that measures the overall performance of a classifier is the “Area Under ROC” or AUROC (or just AUC) value.
- As the name suggests, it is simply the area measured under the ROC curve.
- A higher value of AUC represents a better classifier. The AUC of the practical learner above is 90% which is a good score.
- The AUC of the random classifier = 0.5 and that of perfect learner = 1.

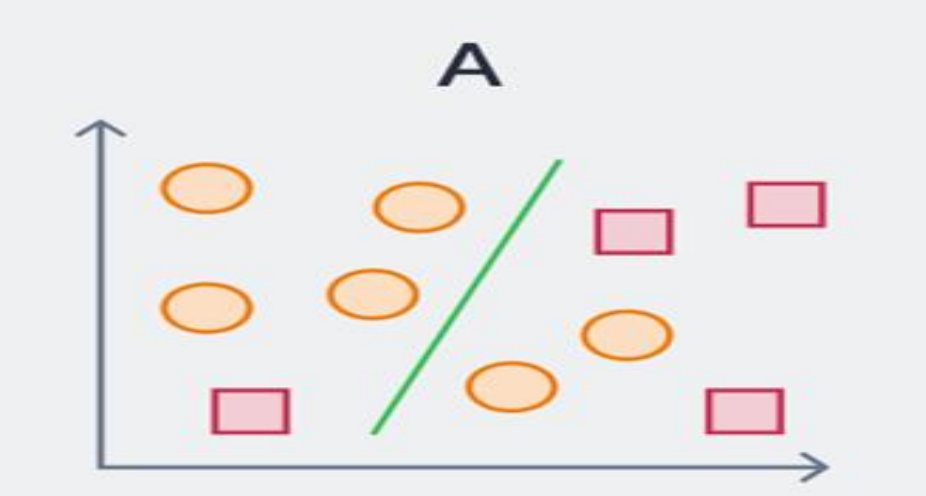
Receiver Operating Characteristics (ROC) curve

- We have mentioned earlier that an ROC curve is a plot for displaying the tradeoff between “true positive rate” and “false positive rate” of a classifier at different threshold settings.
- Now, what do we mean by “thresholds” in the context of ROC curves?

  - Different thresholds represent the different possible classification boundaries of a model. Let us understand this with an example.
- Suppose we have a binary class dataset with 4 positive class samples and 6 negative class samples.

ROC Curve

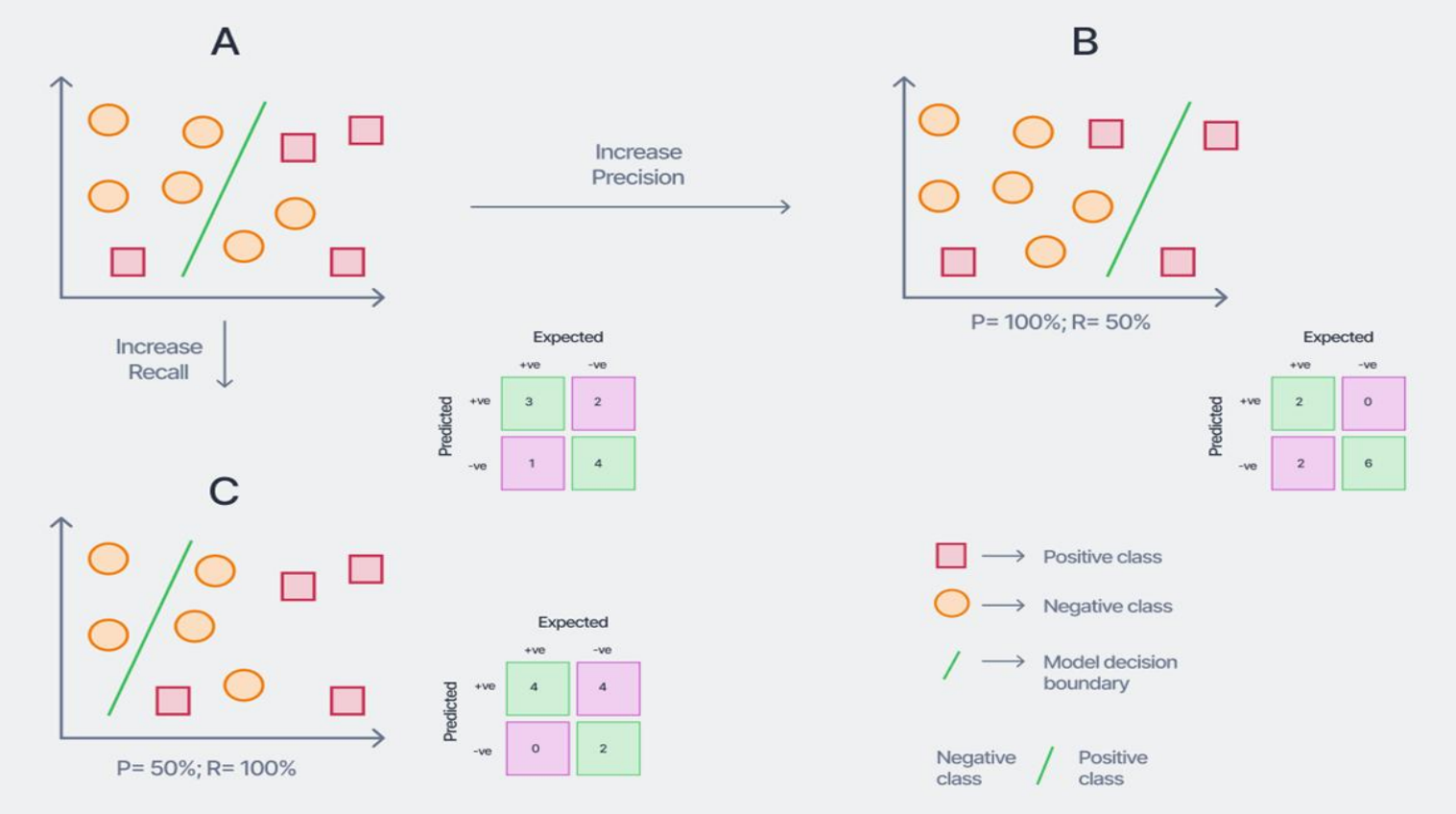
- Let the model decision boundary is as shown by the green line in case (A) below.
- Let, the RIGHT side of the decision boundary depicts the positive class, and the LEFT side depicts the negative class.



- Now, this decision boundary threshold can be changed to arrive at case (B), where the precision is 100% (but recall is 50%), or to case (C) where the recall is 100% (but precision is 50%).

ROC Curve

- Now, this decision boundary threshold can be changed to arrive at case (B), where the precision is 100% (but recall is 50%), or to case (C) where the recall is 100% (but precision is 50%). The corresponding confusion matrices are shown.



ROC Curve

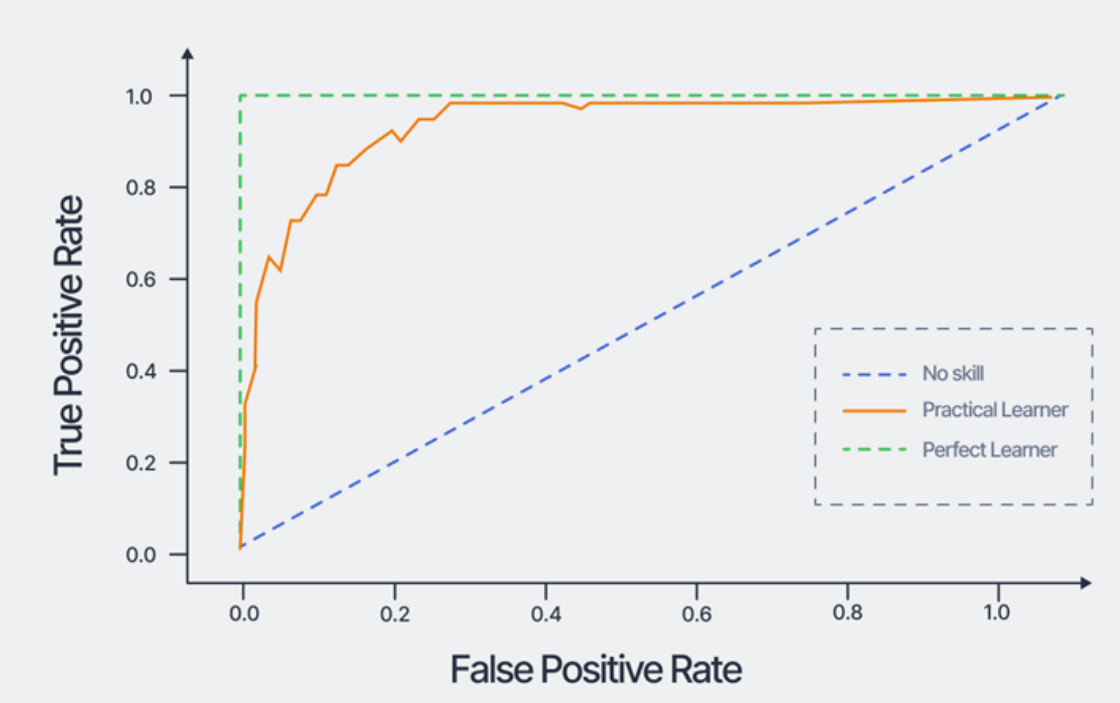
- The TPR and FPR values for these three scenarios with the different thresholds are thus as shown below.

	(A)	(B)	(C)
TPR	75%	50%	100%
FPR	33%	0%	67%

• To know in details about how to generate ROC curve, go through the book, "Introduction to Data Mining" by TAN, STEINBACH, AND VIPIN KUMAR

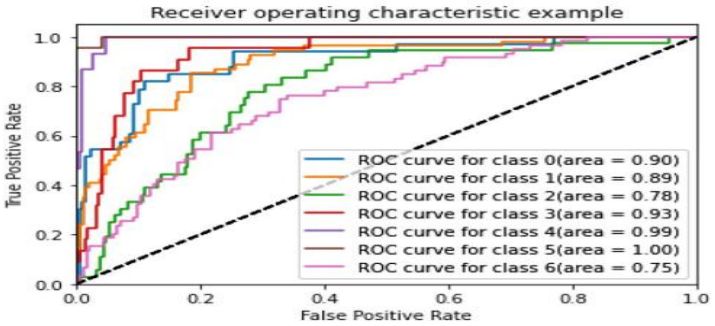
ROC Curve

- Using these values, the ROC curve can be plotted. An example of a ROC curve for a binary classification problem (with randomly generated samples) is shown below.

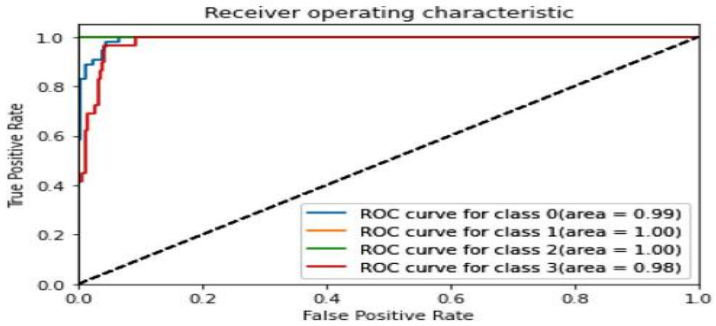


ROC Curve

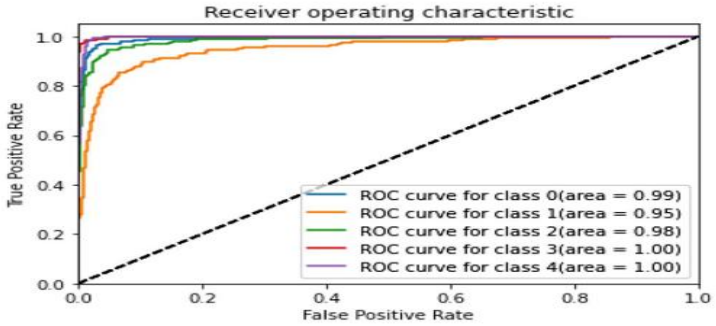
- For multi-class datasets, the ROC curves are plotted by dissolving the confusion matrix into one-vs-all matrices.



(a) Herlev Pap Smear dataset



(b) Mendeley LBC dataset



(c) SIPaKMeD Pap Smear dataset


Thank You